

Consumer Setup - Part2

In Part-1, already seen how to setup basic consumer.



Till now in Basic Consumer setup, we have seen, how to create a Consumer and how it can read from a Topic.

But what if:

- 1 consumer request to join more than 1 topic?
- multiple consumers in a group request to join different topic?
- multiple consumers in a group request to join same topic?

How to achieve this? And this is where things get interesting

In Basic Consumer setup:

```
1 @Component
2 public class OrderEventListener {
3
4     @KafkaListener(topics="order-events")
5     public void consume (Order order) {
6         //business logic
7         System.out.println("Received:" + order.getId());
8     }
9 }
```

```

8     }
9 }
10

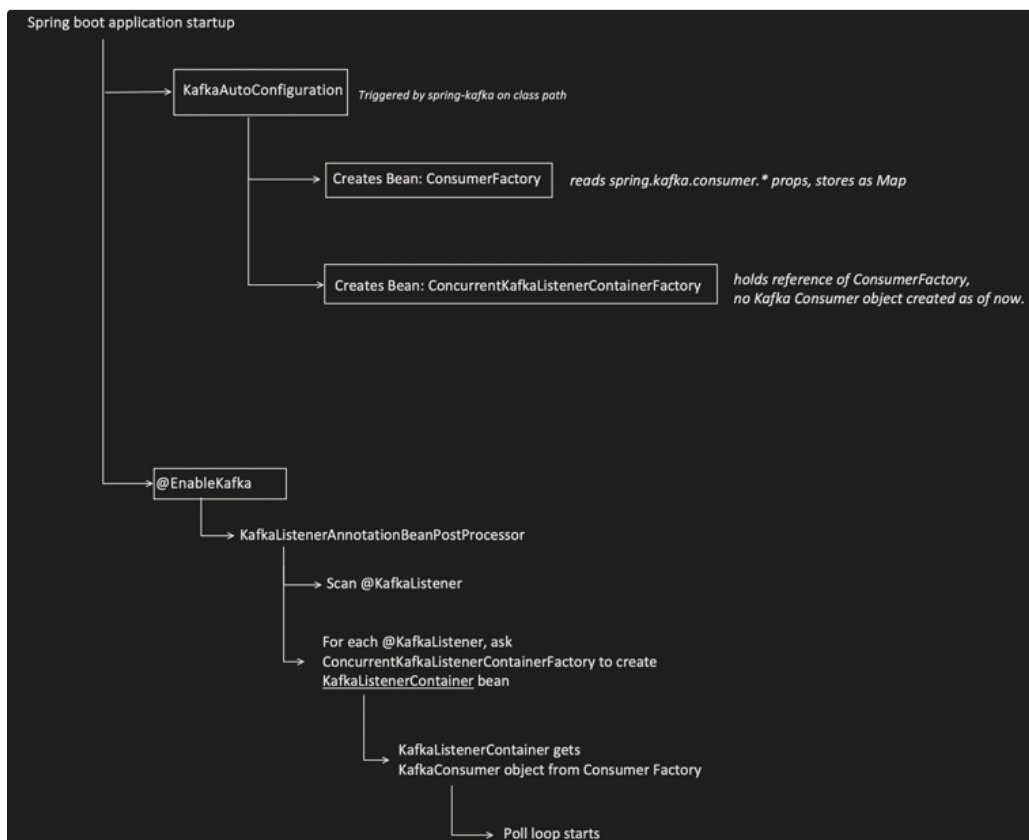
```

```

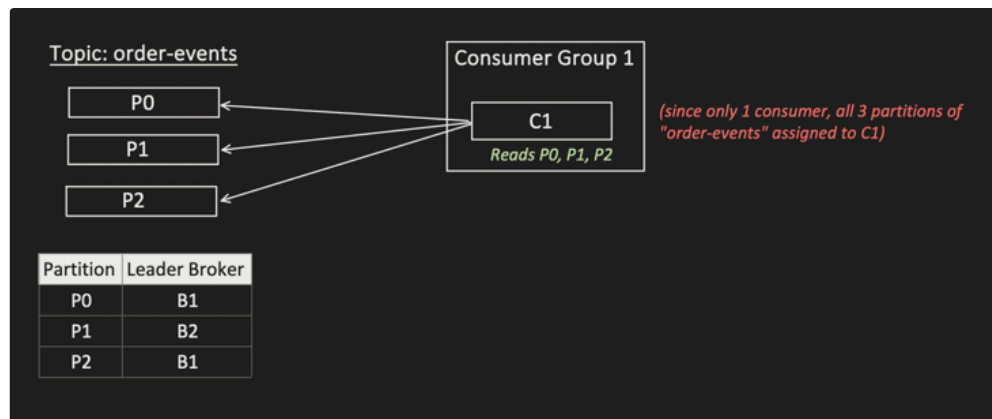
1 server.port=8082
2 spring.application.name=kafka-consumer-service
3
4 spring.kafka.bootstrap-servers=localhost:9092,localhost:9192
5
6 spring.kafka.consumer.group-id=order-consumer-group
7
8 spring.kafka.consumer.key-
9 deserializer=org.apache.kafka.common.serialization.StringDeserializer
10
11 spring.kafka.consumer.value-
12 deserializer=org.springframework.kafka.support.serializer.JsonDeserializer
13
14 spring.kafka.consumer.properties.spring.json.value.default.type=com.ed
15 a.consumer.model.Order

```

What we saw before is:



Let say: "order-events" topic has 3 partition (P0, P1 and P2), so with above Basic consumer setup, what we achieve is:

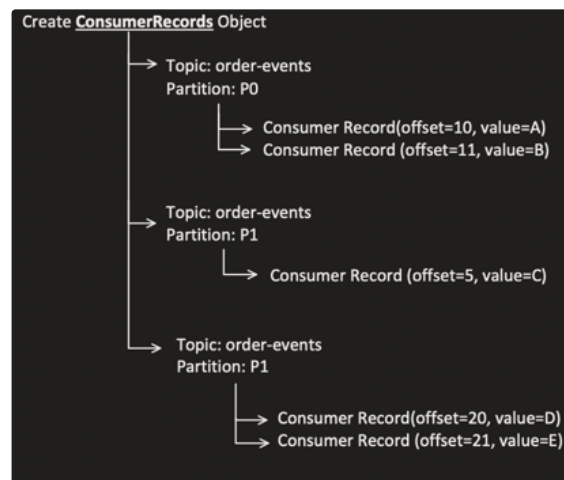
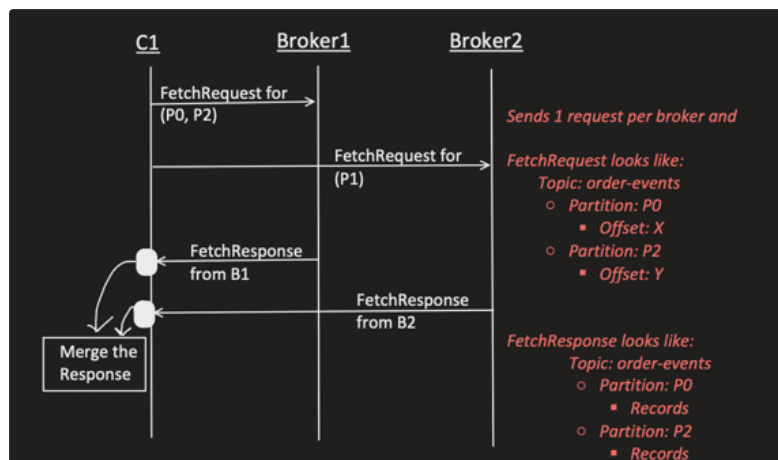


Internally, each consumer might be doing:

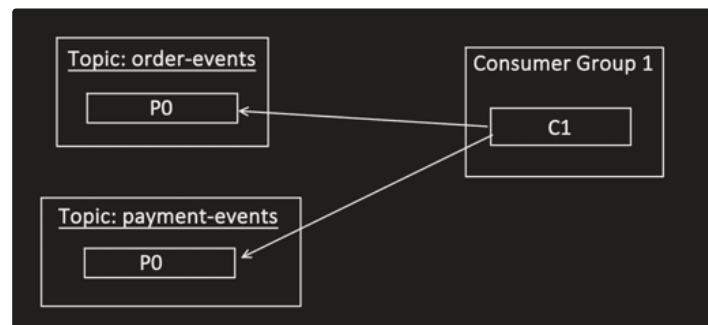
```

1 while (true) {
2     ConsumerRecords records = consumer.poll();
3
4     for (ConsumerRecord record : records) {
5         //business logic
6     }
7 }
8

```



1st: how to set up, 1 consumer request to join more than 1 topic?



Approach 1 : Manually taking care of Json to object mapping

application.properties

```
1 server.port=8082
2 spring.application.name=kafka-consumer-service
3
4 spring.kafka.bootstrap-servers=localhost:9092,localhost:9192
5
6 spring.kafka.consumer.group-id=order-consumer-group
7
8 spring.kafka.consumer.key-
9   deserializer=org.apache.kafka.common.serialization.StringDeserializer
10   <!--
11     Now this consumer handles more than 1 topic, so I will
12     handle the
13     Deserialization like if its order-events -> convert it to Order
14     object
15     If its payment-events -> convert it to Payment object.
16     So, that's why here I am using StringDeserializer. As I need to
17     control the deserialization.
18     -->
19   spring.kafka.consumer.value-
20     deserializer=org.apache.kafka.common.serialization.StringDeserializer
```

```
1 @Component
2 public class OrderEventListener {
3
4   @Autowired
5   ObjectMapper objectMapper;
6   /*
7    Like this we can provide the list of topics which a Consumer wanted
8    to subscribe.
9    */
10  @KafkaListener(topics= {"order-events", "payment-events"})
11  /*
12   here in parameter, I accept the record as it is and
13   Deserialization task I am taking care of.
14   */
15  public void consume (ConsumerRecord<String, String> record) {
```

```

15         If("order-events".equals(record.topic())) {
16
17             Order order =
18             objectMapper.readValue(record.value(), Order.class);
19             // business logic for order
20         }
21         else if("payment-events".equals(record.topic())) {
22
23             Payment payment =
24             objectMapper.readValue(record.value(), Payment.class);
25             // business logic for order
26         }
27     }
28 }

```

Approach 2: AUTO Mapping Json to object conversion

But before that first understand the very important **JSON DESERIALIZATION FLOW**, where many of us get confused:

At PRODUCER side (during JSON Serialization):

```

1  @Service
2  public class OrderProducerService {
3
4      @Autowired
5      private KafkaTemplate<String, Order> kafkaTemplate;
6
7      public void sendWithKey(Order order) {
8
9          String key = order.getOrderid();
10
11          kafkaTemplate.send("order-events", key, order);
12
13      }
14  }
15

```

application.properties

```

1  .
2  .
3  .
4  #-----SERIALIZER-----
5  # Key needs to be String
6  spring.kafka.producer.key-
7  serializer=org.apache.kafka.common.serialization.StringSerializer
8
9  # Value: JsonSerializer, auto-converts Java objects to JSON bytes
10 spring.kafka.producer.value-
11 serializer=org.springframework.kafka.support.serializer.JsonSerializer
12 /*
13 By-default this (json.add.type.headers) is true only
14 */

```

```
13 spring.kafka.producer.properties.spring.json.add.type.headers=true
14 .
15 .
16 .
17
```

With this, when producer send the event:

```
1 kafkaTemplate.send("order-events", order);
```

It will also add `_TypeId_ = com.eda.producer.model.Order` in Event header

```
offset: 0 CreateTime: 1773812798798 keySize: 3 valueSize: 112 sequence: 0 headerKeys: [__TypeId...] key: 0-2 payload: {"orderId": 248000000, "lastOffset": 1, "lastCount": 1, "baseSequence": 0, "lastSequence": 0, "producerId": 0, "producerEpoch": 0, "partitionLeaderEpoch": 0, "isTransactional": 0, "baseMagic": 2, "compressionCode": "snappy", "crc": 2988695115, "isvalid": true}
```

But, its not always 100% sure that Producer will add "_TypeID_" or not, even when its true (which is default)

Like till now, we have seen, headerKeys were empty , even though

```
1 spring.kafka.producer.properties.spring.json.add.type.headers=true
   (Default)
```

```
$ shrayan@shrayans-MacBook-Pro-2 kafka_2.13-4.2.0 % bin/kafka-dump-log.sh --deep-iteration --print-data-log \
--files /tmp/broker-logs/order-events/0/00000000000000000000.log
Dumping /tmp/broker-logs/order-events/0/00000000000000000000.log
Log starting offset: 0
baseOffset: 0 lastOffset: 0 count: 1 baseSequence: 0 lastSequence: 0 producerId: 1800 producerEpoch: 0 partitionLeaderEpoch: 0 isTransactional: false isControl: false deleteHorizonMs: Optio
nallog.empty.partition: 0 CreateTime: 1739574261482 size: 135 magic: 2 compressionCodecName: CRC_64_KRATCHEVICHS invalid: true
offset: 0 CreateTime: 1739574261482 keySize: 7 valueSize: 42 sequence: 0 headerKeys: [] key: O-32700 payload: {"orderId": "O-32700","productId":"PROD-3"}
shrayan@shrayans-MacBook-Pro-2 kafka_2.13-4.2.0 %
```

Why because we are using `KafkaTemplate<String, Order> kafkaTemplate`. Specific value type `Order` is used, so it did not add `_TypeId_`

```

1  @Service
2  public class OrderProducerService {
3
4  @Autowired
5  private KafkaTemplate<String, Order> kafkaTemplate;
6
7  public void sendWithKey(Order order) {
8
9      String key = order.getOrderid();
10
11      kafkaTemplate.send("order-events", key, order);
12
13  }
14  }
15

```

But if I use `KafkaTemplate<String, Object> kafkaTemplate`.

I used `Object` instead of `Order`, Now you will notice that `Producer` will add `_TypeId_` in header.

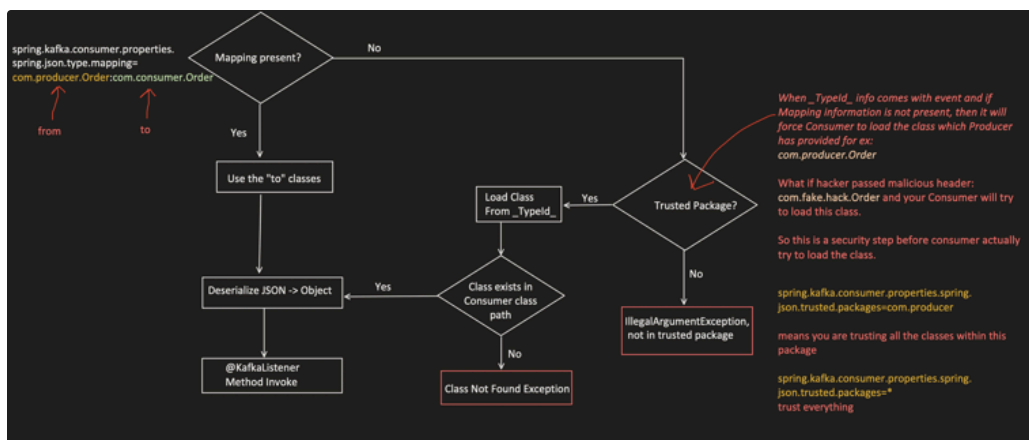
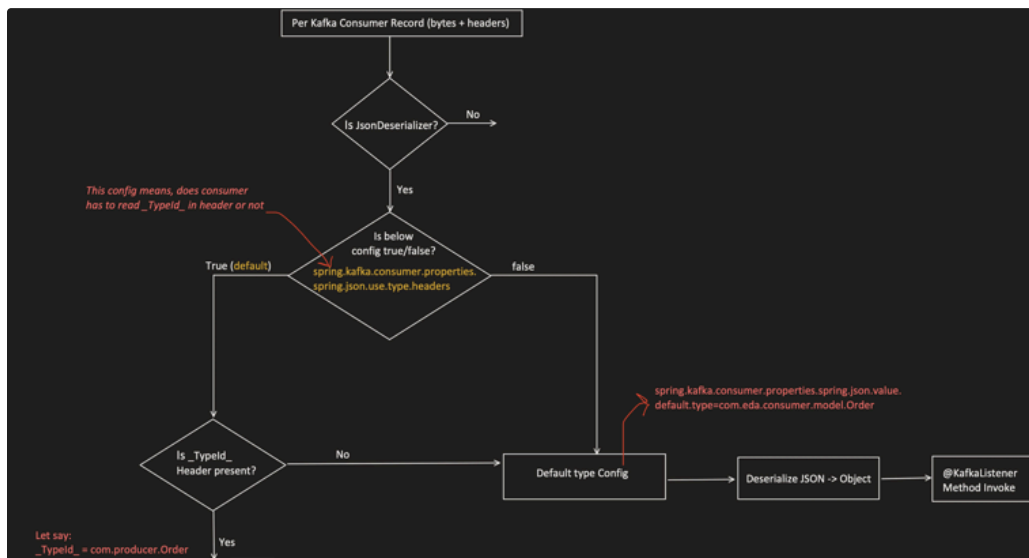
That's internal logic of `Kafka` producer client, but we have to understand that when `Value` uses `JSON SERIALIZER` then by-default below config is true:

`spring.kafka.producer.properties.spring.json.add.type.headers=true`

So producer will add `_TypeId_` in event header.

`_TypeId_ = com.eda.producer.model.Order`

At CONSUMER side (during JSON Deserialization):



Now flow is clear, lets move back to AUTO Mapping (Json to object conversion)

Producer:

```
1 @Service
2 public class ProducerService {
3
4     @Autowired
5     private KafkaTemplate<String, Object> kafkaTemplate;
6
7     public void sendOrder(Order order) {
8         String key = order.getOrderid();
9
10        kafkaTemplate.send("order-events", key, order);
11    }
12
13    public void sendPayment(Payment payment) {
14        String key = payment.getPaymentId();
15
16        kafkaTemplate.send("payment-events", key, payment);
17    }
18 }
```

Producer will add **_TypeId_** in the event header

Consumer:

```
1 server.port=8082
2 spring.application.name=kafka-consumer-service
3
4 spring.kafka.bootstrap-servers=localhost:9092,localhost:9192
5
6 spring.kafka.consumer.group-id=order-consumer-group
7
8 spring.kafka.consumer.key-
9 deserializer=org.apache.kafka.common.serialization.StringDeserializer
10
11 spring.kafka.consumer.value-
12 deserializer=org.springframework.kafka.support.serializer.JsonDeserial
13 izer
14
15 spring.kafka.consumer.properties.spring.json.type.mapping=
16 com.eda.producer.model.Order:com.eda.consumer.model.Order,
17 com.eda.producer.model.Payment:com.eda.consumer.model.Payment
18
19 spring.kafka.consumer.properties.spring.json.value.default.type=com.ed
20 a.consumer.model.Order
```

spring.kafka.consumer.properties.spring.json.type.mapping=

com.eda.producer.model.Order:com.eda.consumer.model.Order,

com.eda.producer.model.Payment:com.eda.consumer.model.Payment

These are nothing but only the classes in POJO

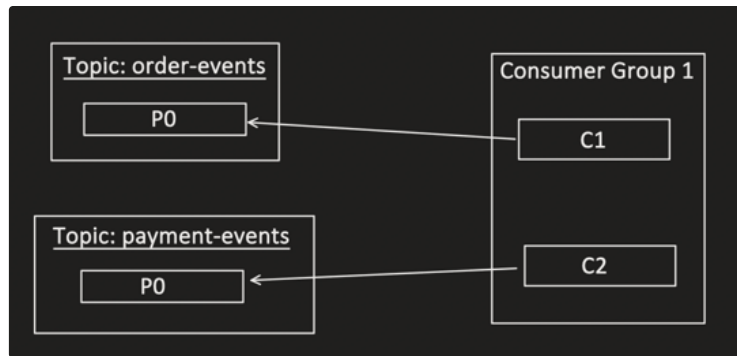
Ex:-

```
1 package com.eda.consumer.model;
2
3 public class Order {
4
5     private String orderId;
6     private String customerId;
7     private String productId;
8     private Integer quantity;
9     private Double totalAmount;
10    private String status;
11
12    //getters and setters
13 }
```

```
1 package com.eda.consumer.model;
2
3 public class Payment {
4
5     private String paymentId;
6     private String customerId;
7     private Double totalAmount;
8     private String status;
9
10    //getters and setters
11 }
```

```
1 @Component
2 public class OrderEventListener {
3
4     @KafkaListener(topics= {"order-events", "payment-events"})
5     public void consume (ConsumerRecord<String, String> record) {
6         if("order-events".equals(record.topic())) {
7             /*
8              * Now no manual
9              * mapping is required
10             */
11             Order order = (Order) record.value();
12             // business logic for order
13         }
14         else if("payment-events".equals(record.topic())) {
15             Payment payment = (Payment) record.value();
16             // business logic for order
17         }
18     }
19 }
20
21
22 }
```

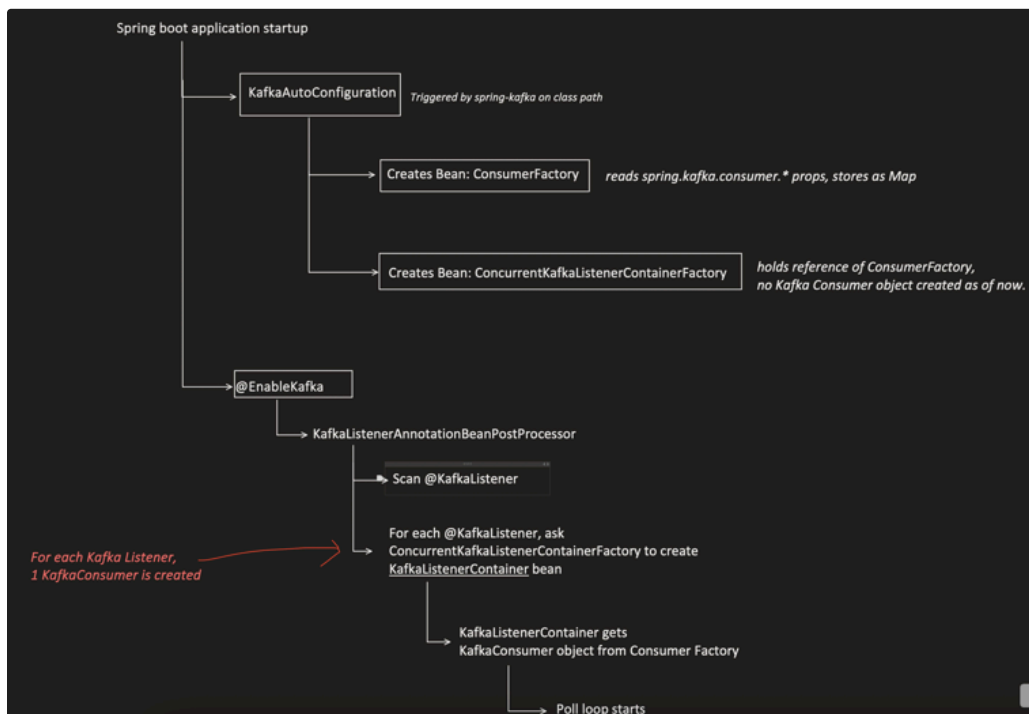
2nd: how to set up, multiple consumers in a group request to join different topic?



Approach 1: 2 different consumer application

The most simple is, create 2 different Consumer application and whatever we have seen in basic consumer setup, that will work, as it creates 1 Consumer per application.

Approach 2: in 1 application itself we need 2 consumer



There are 2 ways:-

TypeId + Mapping

Use Header = false

Now in _TypeId_ + Mapping

```

1 @Component
2 public class EventListener {
3
4     @KafkaListener(topics="order-events")
5     public void consumeOrder (Order order) {
6         //business logic
7         System.out.println("Received:" + order.getOrderid());
8     }
9
10    @KafkaListener(topics="payment-events")
11    public void consumePayment (Payment payment) {
12        //business logic
13        System.out.println("Received:" + payment.getPaymentId());
14    }
15 }
16

```

application.properties

```

1 server.port=8082
2 spring.application.name=kafka-consumer-service
3
4 spring.kafka.bootstrap-servers=localhost:9092,localhost:9192
5
6 spring.kafka.consumer.group-id=order-consumer-group
7
8 spring.kafka.consumer.key-
9 deserializer=org.apache.kafka.common.serialization.StringDeserializer
10
11 spring.kafka.consumer.value-
12 deserializer=org.springframework.kafka.support.serializer.JsonDeserial
13 izer
14
15 spring.kafka.consumer.properties.spring.json.type.mapping=
16 com.eda.producer.model.Order:com.eda.consumer.model.Order,
17 com.eda.producer.model.Payment:com.eda.consumer.model.Payment
18
19 spring.kafka.consumer.properties.spring.json.value.default.type=com.ed
20 a.consumer.model.Orde

```

This works fine and recommended.

Use Header = false

```

1 @Component
2 public class EventListener {
3
4     @KafkaListener(topics="order-events")
5     public void consumeOrder (Order order) {
6         //business logic
7         System.out.println("Received:" + order.getOrderid());
8     }
9
10    @KafkaListener(topics="payment-events")
11    public void consumePayment (Payment payment) {
12        //business logic

```

```

13         System.out.println("Received:" + payment.getPaymentId());
14     }
15 }
16

```

application.properties

```

1 server.port=8082
2 spring.application.name=kafka-consumer-service
3
4 spring.kafka.bootstrap-servers=localhost:9092,localhost:9192
5
6 spring.kafka.consumer.group-id=order-consumer-group
7
8 spring.kafka.consumer.key-
9   deserializer=org.apache.kafka.common.serialization.StringDeserializer
10
11 spring.kafka.consumer.value-
12   deserializer=org.springframework.kafka.support.serializer.JsonDeserial
13   izer
14
15 spring.kafka.consumer.properties.spring.json.use.type.headers=false
16
17 spring.kafka.consumer.properties.spring.json.value.default.type=com.eda
18   a.consumer.model.Order
19

```

This will not work, when payment event is sent:

- Consumer will consume it
- Use header = false
- Go to default type config, which is

spring.kafka.consumer.properties.spring.json.value.default.type=com.eda.consumer.model.Order

Exception will come:

```

Caused by: org.springframework.messaging.converter.MessageConversionException: Cannot convert
from [com.eda.consumer.model.Order] to [com.eda.consumer.model.Payment] for GenericMessage
[payload=Order{orderId='null', customerId='CUST-2', productId='null', quantity=null, totalAmount=99.99,
status='CREATED'}, headers={kafka_offset=0,

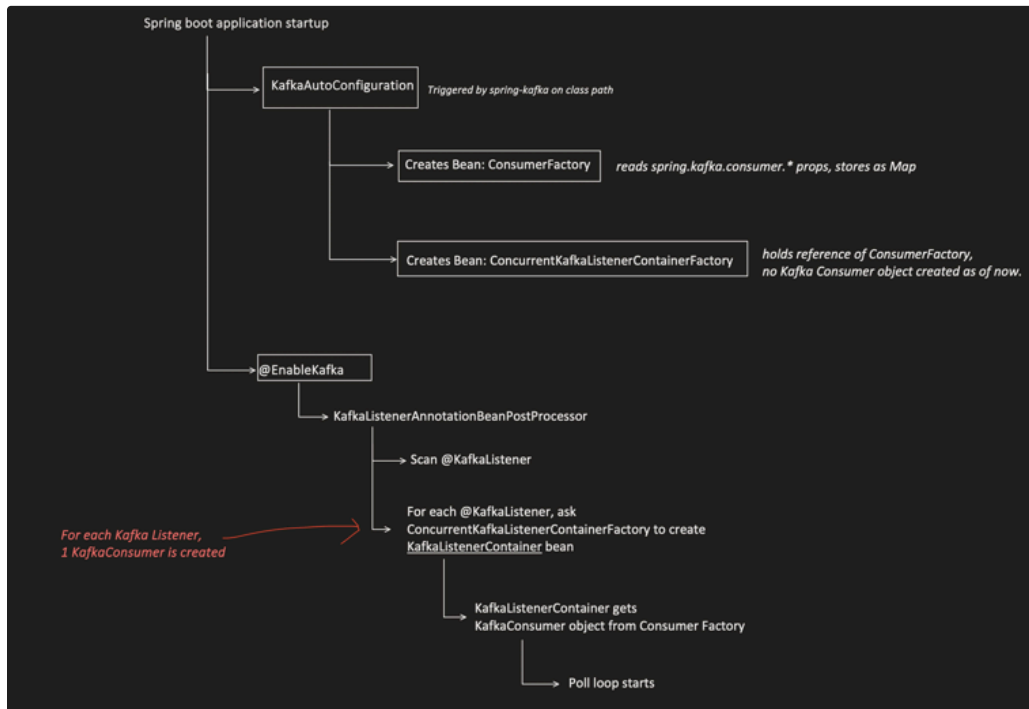
```

How to resolve this?

This issue comes because we have only **1 consumer Factory object** which is created from properties defined in application.properties file.

So in this **ConsumerFactory** Object, "default Type" is what we set in application.properties i.e .

**spring.kafka.consumer.properties.spring.json.value.default.type=com.eda.consumer.mode
I.Order**



And now Same **ConsumerFactory** object is used to create both KafkaConsumer:

- KafkaConsumer for Order events.
- KafkaConsumer for Payment event.

And since same ConsumerFactory object is used, both KafkaConsumer has "default Type" =

**spring.kafka.consumer.properties.spring.json.value.default.type=com.eda.consumer.mode
I.Order**

So if we get this, then solution is very simple, have different ConsumerFactory for each event type with default type is specific to each event.

Kafka Consumer Config

```
1 package com.eda.consumer.config;
```

```

3 import com.eda.consumer.model.Order;
4 import com.eda.consumer.model.Payment;
5 import org.springframework.boot.autoconfigure.kafka.KafkaProperties;
6 import org.springframework.context.annotation.Bean;
7 import org.springframework.context.annotation.Configuration;
8 import
  org.springframework.kafka.config.ConcurrentKafkaListenerContainerFacto
  ry;
9 import org.springframework.kafka.core.ConsumerFactory;
10 import org.springframework.kafka.core.DefaultKafkaConsumerFactory;
11 import org.springframework.kafka.support.serializer.JsonDeserializer;
12 import java.util.Map;
13
14 @Configuration
15     public class ConsumerConfig {
16
17 @Bean
18     public ConsumerFactory<String, Order>
  orderConsumerFactory(KafkaProperties props) {
19         Map<String, Object> config =
  props.buildConsumerProperties();
20         config.put(JsonDeserializer.VALUE_DEFAULT_TYPE,
  Order.class.getName());
21         return new DefaultKafkaConsumerFactory<>(config);
22     }
23
24     @Bean
25     public ConsumerFactory<String, Payment>
  paymentConsumerFactory(KafkaProperties props) {
26         Map<String, Object> config = props.buildConsumerProperties();
27         config.put(JsonDeserializer.VALUE_DEFAULT_TYPE,
  Payment.class.getName());
28         return new DefaultKafkaConsumerFactory<>(config);
29     }
30
31     @Bean
32     public ConcurrentKafkaListenerContainerFactory<String, Order>
  orderKafkaListenerFactory(ConsumerFactory<String, Order>
  orderConsumerFactory) {
33         ConcurrentKafkaListenerContainerFactory<String, Order> factory
  = new ConcurrentKafkaListenerContainerFactory<>();
34         factory.setConsumerFactory(orderConsumerFactory);
35         return factory;
36     }
37
38     @Bean
39     public ConcurrentKafkaListenerContainerFactory<String, Payment>
  paymentKafkaListenerFactory(ConsumerFactory<String, Payment>
  paymentConsumerFactory) {
40         ConcurrentKafkaListenerContainerFactory<String, Payment>
  factory = new ConcurrentKafkaListenerContainerFactory<>();
41         factory.setConsumerFactory(paymentConsumerFactory);
42         return factory;
43     }
44

```

```

1 @Component
2 public class EventListener {
3
4     @KafkaListener(topics="order-events",
  containerFactory="orderKafkaListenerFactory")
5     public void consumeOrder (Order order) {
6         //business logic
7         System.out.println("Received:" + order.getOrderid());
8     }
9

```

```

10 @KafkaListener(topics="payment-events",
    containerFactory="paymentKafkaListenerFactory")
11 public void consumePayment (Payment payment) {
12     //business logic
13     System.out.println("Received:" + payment.getPaymentId());
14 }
15 }
16

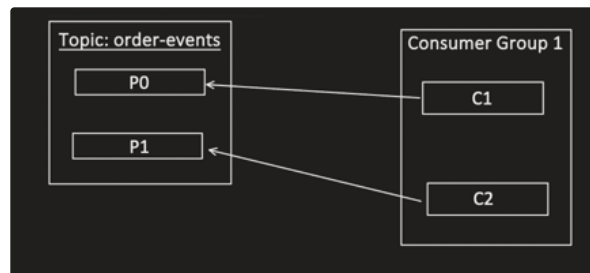
```

```

1 server.port=8082
2 spring.application.name=kafka-consumer-service
3
4 spring.kafka.bootstrap-servers=localhost:9092,localhost:9192
5
6 spring.kafka.consumer.group-id=order-consumer-group
7
8 spring.kafka.consumer.key-
deserializer=org.apache.kafka.common.serialization.StringDeserializer
9
10 spring.kafka.consumer.value-
deserializer=org.springframework.kafka.support.serializer.JsonDeserial
izer
11
12 spring.kafka.consumer.properties.spring.json.use.type.headers=false
13
14 spring.kafka.consumer.properties.spring.json.value.default.type=com.ed
a.consumer.model.Order
15

```

3rd: how to set up, multiple consumers in a group request to join same topic?



```

1 @Component
2 public class EventListener {
3
4     @KafkaListener(topics="order-events",)
5     public void consumeOrder (Order order) {
6         //business logic
7         System.out.println("Received:" + order.getOrderid());
8     }
9 }
10

```

```

1 server.port=8082
2 spring.application.name=kafka-consumer-service
3
4 spring.kafka.bootstrap-servers=localhost:9092,localhost:9192
5
6 spring.kafka.consumer.group-id=order-consumer-group
7
8 spring.kafka.consumer.key-
deserializer=org.apache.kafka.common.serialization.StringDeserializer

```

```

9
10 spring.kafka.consumer.value-
    deserializer=org.springframework.kafka.support.serializer.JsonDeserial
    izer
11
12 spring.kafka.consumer.properties.spring.json.use.type.headers=false
13
14 spring.kafka.consumer.properties.spring.json.value.default.type=com.ed
    a.consumer.model.Order
15
16 spring.kafka.listener.concurrency=3
17

```

